Cómo cambiar de API en la app Clima

Paso a paso sobre tu código (`com.example.clima`). Ejemplo completo: cambiar de **WeatherAPI.com** a **OpenWeatherMap**.

Antes de empezar: cómo está armada tu app

Tu app está separada en capas. Cuando cambias de API, **solo tocas la capa `datos/`** y algunos textos. Las entidades (`entidad/`), el ViewModel y casi toda la UI quedan igual.

```
Internet → ApiClient (URL base) → WeatherApiService (endpoints) → DTOs (forma del JSON)
              → Mapper (traduce) → Entidades (no cambian) → ViewModel → UI
```

Archivo	Qué hace	¿Cambia?
`apikey.properties`	Guarda tu clave secreta	Sí, pones la clave nueva
`app/build.gradle.kts`	Pasa la clave a `BuildConfig`	Opcional (solo si renombras la clave)
`datos/ApiClient.kt`	Configura Retrofit con la URL base	**Sí**
`datos/WeatherApiService.kt`	Endpoints y parámetros	**Sí**
`datos/dto/ClimaDtos.kt`	Copia la forma del JSON	**Sí**
`datos/mapper/ClimaMapper.kt`	Convierte DTO en entidad	**Sí**
`datos/ClimaRepository.kt`	Llama a la API y maneja los errores	**Sí**
`presentacion/ClimaViewModel.kt`	Usa el repositorio	Solo si renombras la clave
`ui/ClimaApp.kt`	Pantallas	Solo textos e íconos (líneas 129 y 214)
`entidad/*.kt`	Modelo de la app	**No**
`test/ClimaRepositoryTest.kt` y `clima.json`	Pruebas	**Sí**

Paso 0 · Preparación

☐ Haz una **copia de seguridad** de la carpeta del proyecto (o un commit en git), para poder volver atrás.

☐ Regístrate en la nueva API (por ejemplo `openweathermap.org`) y copia tu **clave (API key)**.

☐ Lee su documentación y anota: **URL base**, **endpoints**, **cómo se manda la clave** (parámetro `?appid=`, `?key=` o un header) y **qué devuelve cuando hay error**.

☐ Prueba la API en el navegador y **guarda el JSON de respuesta**, que lo usarás para los DTOs y los tests:

```
https://api.openweathermap.org/data/2.5/weather?q=La%20Paz&appid=TU_CLAVE&units=metric&lang=es
https://api.openweathermap.org/data/2.5/forecast?q=La%20Paz&appid=TU_CLAVE&units=metric&lang=es
```

> **Ojo**
> Una clave nueva de OpenWeatherMap puede tardar hasta un par de horas en activarse. Mientras tanto devuelve error 401. No es un fallo de tu código.

Compara qué datos da cada API

Tus entidades necesitan ciertos datos. Revisa si la nueva API los trae y, si no, decide qué valor poner:

Entidad (no cambia)	WeatherAPI (antes)	OpenWeatherMap (después)
`temp_c`	`current.temp_c`	`main.temp` (con `units=metric`)
`feelslike_c`	`current.feelslike_c`	`main.feels_like`
`humidity`	`current.humidity`	`main.humidity`
`wind_kph`	`current.wind_kph`	`wind.speed` está en **m/s**: multiplica por 3.6
`precip_mm`	`current.precip_mm`	`rain.1h` (puede no venir: usa 0.0)
`uv`	`current.uv`	**No viene** en el plan gratis: usa 0.0
`condition`	`condition.text/icon/code`	`weather[0].description/icon/id`
`region`	`location.region`	**No viene**: usa `""`
`localtime`	`location.localtime`	Se calcula con `timezone` (desfase en segundos)
Pronóstico por día	`forecast.forecastday[]`	`list[]` en **bloques de 3 horas**: hay que agrupar por fecha
`daily_chance_of_rain`	entero 0–100	`pop` decimal 0–1: multiplica por 100

Paso 1 · Poner la clave nueva

Copia `apikey.properties.example` como `apikey.properties` en la raíz del proyecto y pon tu clave:

`apikey.properties`

```
WEATHER_API_KEY=tu_clave_de_openweathermap
```

Recomendado: deja el nombre `WEATHER_API_KEY`

Es un nombre genérico y así no tienes que tocar `build.gradle.kts` ni `ClimaViewModel.kt`. Si aun así lo renombras (por ejemplo a `OWM_API_KEY`), cámbialo en **todos** estos lugares:

- `apikey.properties` y `apikey.properties.example`
- `app/build.gradle.kts`, líneas 12–13 (`getProperty` y `getenv`) y línea 25 (`buildConfigField`)
- `ClimaViewModel.kt`, líneas 25 y 28 (`BuildConfig.WEATHER_API_KEY`)

Después pulsa **Sync Project with Gradle Files** en Android Studio.

Paso 2 · Cambiar la URL base

`datos/ApiClient.kt`

```
Retrofit.Builder()
    .baseUrl("https://api.openweathermap.org/data/2.5/")   // ← antes:
https://api.weatherapi.com/v1/
    .client(client)
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(WeatherApiService::class.java)
```

Regla de Retrofit

La URL base **debe terminar en `/`**. Si no, la app falla al arrancar con `IllegalArgumentException: baseUrl must end in /`.

Paso 3 · Cambiar los endpoints

OpenWeatherMap usa **dos** endpoints: uno para el clima actual y otro para el pronóstico. La clave va en el parámetro `appid`.

`datos/WeatherApiService.kt`

```
package com.example.clima.datos

import com.example.clima.datos.dto.OwmActualDto
import com.example.clima.datos.dto.OwmPronosticoDto
import retrofit2.http.GET
import retrofit2.http.Query

interface WeatherApiService {
    @GET("weather")
    suspend fun obtenerActual(
        @Query("appid") key: String,
        @Query("q") ciudad: String,
        @Query("units") unidades: String = "metric",
        @Query("lang") idioma: String = "es"
    ): OwmActualDto

    @GET("forecast")
    suspend fun obtenerPronostico(
        @Query("appid") key: String,
        @Query("q") ciudad: String,
        @Query("units") unidades: String = "metric",
        @Query("lang") idioma: String = "es"
    ): OwmPronosticoDto
}
```

Si la API pide la clave en un header

Usa `@Header("X-Api-Key") key: String` en lugar de `@Query`.

Paso 4 · Crear los DTOs nuevos

Los DTOs copian la **forma exacta del JSON**. En `@SerializedName` va el nombre del campo tal como aparece en el JSON. El nombre de la variable en Kotlin lo eliges tú. Reemplaza el contenido de `ClimaDtos.kt`:

`datos/dto/ClimaDtos.kt`

```
package com.example.clima.datos.dto

import com.google.gson.annotations.SerializedName

// Modelos de los datos que llegan desde OpenWeatherMap.
data class OwmCondicionDto(
    @SerializedName("id") val codigo: Int,
    @SerializedName("description") val descripcion: String,
    @SerializedName("icon") val icono: String
)

data class OwmCoordDto(
    @SerializedName("lat") val latitud: Double,
    @SerializedName("lon") val longitud: Double
)

data class OwmMainDto(
    @SerializedName("temp") val temperatura: Double,
    @SerializedName("feels_like") val sensacion: Double,
    @SerializedName("temp_min") val minima: Double,
    @SerializedName("temp_max") val maxima: Double,
    @SerializedName("humidity") val humedad: Int
)

data class OwmVientoDto(@SerializedName("speed") val velocidad: Double)

data class OwmLluviaDto(@SerializedName("1h") val unaHora: Double?)

data class OwmSysDto(@SerializedName("country") val pais: String?)

data class OwmActualDto(
    @SerializedName("name") val nombre: String,
    @SerializedName("coord") val coord: OwmCoordDto,
    @SerializedName("weather") val condiciones: List<OwmCondicionDto>,
    @SerializedName("main") val principal: OwmMainDto,
    @SerializedName("wind") val viento: OwmVientoDto,
    @SerializedName("rain") val lluvia: OwmLluviaDto?,      // no viene si no llueve
    @SerializedName("sys") val sys: OwmSysDto,
    @SerializedName("dt") val fechaUnix: Long,
    @SerializedName("timezone") val zonaHoraria: Int         // segundos respecto a UTC
)

data class OwmBloqueDto(                                      // un bloque de 3 horas
    @SerializedName("dt") val fechaUnix: Long,
    @SerializedName("main") val principal: OwmMainDto,
    @SerializedName("weather") val condiciones: List<OwmCondicionDto>,
    @SerializedName("pop") val probLluvia: Double             // 0.0 a 1.0
)

data class OwmPronosticoDto(
    @SerializedName("list") val bloques: List<OwmBloqueDto>
)
```

Campos que pueden faltar

Gson no respeta la nulabilidad de Kotlin. Si un campo puede no venir en el JSON (como `rain`), decláralo con `?` y usa `?:` en el mapper. Si no, la app falla con `NullPointerException`.

Paso 5 · Escribir el mapper

Aquí está la parte clave: convertir los DTOs de la API nueva en **las mismas entidades de siempre**. Así el resto de la app no se entera del cambio. Reemplaza el contenido de `ClimaMapper.kt`:

`datos/mapper/ClimaMapper.kt`

```
package com.example.clima.datos.mapper

import com.example.clima.datos.dto.*
import com.example.clima.entidad.*
import java.time.Instant
import java.time.ZoneOffset
import java.time.format.DateTimeFormatter
import kotlin.math.roundToInt

private val formato = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")

// Convierte una fecha Unix a la hora local de la ciudad.
private fun horaLocal(unix: Long, desfase: Int): String =
    Instant.ofEpochSecond(unix).atOffset(ZoneOffset.ofTotalSeconds(desfase)).format(formato)

fun OwmCondicionDto.aEntidad(): Condicion = Condicion(
    text = descripcion.replaceFirstChar { it.uppercase() },
    icon = "https://openweathermap.org/img/wn/${icono}@2x.png",   // URL completa del ícono
    code = codigo
)

// Une las dos respuestas (actual + pronóstico) en la entidad que usa la app.
fun combinarEnEntidad(actual: OwmActualDto, pronostico: OwmPronosticoDto): RespuestaClima {
    val zona = actual.zonaHoraria

    // Agrupa los bloques de 3 horas por fecha y se queda con los 3 primeros días.
    val dias = pronostico.bloques
        .groupBy { horaLocal(it.fechaUnix, zona).substring(0, 10) }
        .entries.take(3)
        .map { (fecha, bloques) ->
            DiaPronostico(
                date = fecha,
                day = ResumenDia(
                    maxtemp_c = bloques.maxOf { it.principal.maxima },
                    mintemp_c = bloques.minOf { it.principal.minima },
                    daily_chance_of_rain = (bloques.maxOf { it.probLluvia } * 100).roundToInt(),
                    condition = bloques[bloques.size / 2].condiciones.first().aEntidad()
                )
            )
        }

    return RespuestaClima(
        location = Ubicacion(
            name = actual.nombre,
            region = "",                                        // OWM no da región
            country = actual.sys.pais.orEmpty(),
            localtime = horaLocal(System.currentTimeMillis() / 1000, zona),
            lat = actual.coord.latitud,
            lon = actual.coord.longitud
        ),
        current = ClimaActual(
            temp_c = actual.principal.temperatura,
            feelslike_c = actual.principal.sensacion,
            humidity = actual.principal.humedad,
            wind_kph = actual.viento.velocidad * 3.6,           // m/s → km/h
            precip_mm = actual.lluvia?.unaHora ?: 0.0,
            uv = 0.0,                                           // OWM gratis no da UV
            last_updated = horaLocal(actual.fechaUnix, zona),
            condition = actual.condiciones.first().aEntidad()
        ),
        forecast = Pronostico(forecastday = dias)
```

```
    )
}
```

Por qué funciona

La UI ya sabe mostrar una URL de ícono que empieza por `https://` (`ClimaApp.kt`, línea 216). Como el mapper arma la URL completa, no hay que tocar la pantalla.

Paso 6 · Actualizar el repositorio

Hay que cambiar la llamada (ahora son dos) y los **errores**. Los códigos 1006, 2006, 2007… son propios de WeatherAPI. OpenWeatherMap usa directamente el **código HTTP**.

datos/ClimaRepository.kt

```
package com.example.clima.datos

import com.example.clima.entidad.RespuestaClima
import com.example.clima.datos.mapper.combinarEnEntidad
import com.google.gson.JsonParseException
import kotlinx.coroutines.CancellationException
import retrofit2.HttpException
import java.io.IOException
import java.net.SocketTimeoutException

class ClimaException(message: String) : Exception(message)

class ClimaRepository(private val api: WeatherApiService, private val key: String) {
    suspend fun obtenerClima(ciudad: String): RespuestaClima {
        val consulta = ciudad.trim()
        if (consulta.isEmpty()) throw ClimaException("Escribe una ciudad para buscar.")
        if (key.isBlank()) throw ClimaException(
            "Falta la clave de OpenWeatherMap. Configura apikey.properties y vuelve a ejecutar la
app."
        )
        try {
            val actual = api.obtenerActual(key, consulta)
            val pronostico = api.obtenerPronostico(key, consulta)
            return combinarEnEntidad(actual, pronostico)
        } catch (e: CancellationException) {
            throw e
        } catch (e: HttpException) {
            throw ClimaException(when (e.code()) {
                404 -> "No encontramos esa ciudad. Prueba escribiendo ciudad y país."
                401 -> "La clave de OpenWeatherMap no es válida. Revisa apikey.properties."
                429 -> "Se alcanzó el límite de consultas de tu cuenta OpenWeatherMap."
                else -> "El servicio no pudo responder (HTTP ${e.code()}). Intenta nuevamente."
            })
        } catch (e: SocketTimeoutException) {
            throw ClimaException("La consulta tardó demasiado. Intenta nuevamente.")
        } catch (e: IOException) {
            throw ClimaException("No se pudo conectar. Comprueba tu conexión a Internet.")
        } catch (e: JsonParseException) {
            throw ClimaException("El servicio devolvió datos que no se pudieron interpretar.")
        }
    }
}
```

Mantén los mensajes

Conviene que los textos sigan conteniendo "Falta la clave", "No encontramos", "no es válida", "límite" e "Internet". Los tests buscan esas palabras.

Paso 7 · Ajustes en la UI

☐ `ui/ClimaApp.kt`, línea 129: cambia el texto `"Datos: WeatherAPI.com"` por `"Datos: OpenWeatherMap"`.

☐ `ui/ClimaApp.kt`, línea 214: el ícono de respaldo usa códigos de WeatherAPI (1000 = sol, 1189 = lluvia). Con OpenWeatherMap: 800 = despejado, 500–531 = lluvia. Ejemplo:
`when (condicion.code) { 800 -> "☀"; in 500..531 -> "☂"; else -> "☁" }`

☐ `datos/DatosDemo.kt` (modo demo): usa los códigos 1003 y 1189. Si cambiaste la línea 214, pon códigos de OWM (por ejemplo 803 y 501) para que el demo siga mostrando bien los íconos.

Paso 8 · Actualizar los tests

`ClimaRepositoryTest.kt` simula la API con `MockWebServer`. Hay que cambiar:

☐ **URL del servidor falso** (línea 25): `server.url("/v1/")` → `server.url("/data/2.5/")`.

☐ **Archivos JSON de prueba**: en `src/test/resources/` crea `actual.json` y `pronostico.json` con las respuestas reales que guardaste en el Paso 0.

☐ **Encolar dos respuestas**, porque ahora hay dos llamadas (primero actual, luego pronóstico):

```
server.enqueue(MockResponse().setBody(leer("/actual.json")))
server.enqueue(MockResponse().setBody(leer("/pronostico.json")))
```

☐ **Comprobar la petición** (líneas 43–49): ahora el path es `/data/2.5/weather`, la clave va en `appid` y ya no existe el parámetro `days`.

☐ **Valores esperados** (líneas 36–42): cámbialos por los que tienen tus JSON nuevos.

☐ **Tests de error** (líneas 63–82): usa el formato de OWM, por ejemplo `setResponseCode(404).setBody("""{"cod":"404","message":"city not found"}""")`, y lo mismo con 401 y 429.

☐ `apiQueFalla` (líneas 100–105): implementa los dos métodos nuevos:

```
private fun apiQueFalla(error: Exception) = object : WeatherApiService {
    override suspend fun obtenerActual(key: String, ciudad: String,
        unidades: String, idioma: String): OwmActualDto = throw error
    override suspend fun obtenerPronostico(key: String, ciudad: String,
        unidades: String, idioma: String): OwmPronosticoDto = throw error
}
```

Cambia también el import `RespuestaClimaDto` por `OwmActualDto` y `OwmPronosticoDto`.

Paso 9 · Compilar, probar y verificar

☐ **Build → Rebuild Project.** Si sale *Unresolved reference*, quedó algún import o nombre viejo (`RespuestaClimaDto`, `aEntidad`…).

☐ Corre los tests: clic derecho en `ClimaRepositoryTest` → **Run** (o `gradlew test`).

☐ Ejecuta la app y busca una ciudad real (por ejemplo *La Paz, BO*).

☐ Busca una ciudad que no existe: debe salir "No encontramos esa ciudad".

☐ Activa el modo avión: debe salir "Comprueba tu conexión a Internet".

☐ Revisa que temperatura, viento (en km/h), humedad, íconos y los 3 días del pronóstico se vean bien.

Problemas comunes

Síntoma	Causa probable	Solución
`baseUrl must end in /`	Falta la `/` final	Paso 2
Error 401 / "clave no válida"	La clave está mal o aún no se activa	Revisa `apikey.properties`, haz Sync Gradle y espera a que se active
La app entra en modo demo	`BuildConfig.WEATHER_API_KEY` está vacía	El archivo no está en la raíz o falta Sync Gradle
Error 404 en todas las ciudades	Endpoint mal escrito	Compara con la URL que probaste en el navegador
`NullPointerException` al leer datos	Un campo no vino en el JSON	Hazlo nullable (`?`) en el DTO (Paso 4)
Temperatura de 290 grados	Faltó `units=metric` (llega en Kelvin)	Paso 3
"Datos que no se pudieron interpretar"	El DTO no coincide con el JSON	Compara los `@SerializedName` con el JSON real
Los íconos no aparecen	La URL del ícono está incompleta	Arma la URL completa en el mapper (Paso 5)

Resumen: la receta para cualquier API

1. Probar la API en el navegador y guardar el JSON.
2. Poner la clave en `apikey.properties`.
3. Cambiar la URL base (`ApiClient`).
4. Cambiar endpoints y parámetros (`WeatherApiService`).
5. Crear DTOs con la forma del JSON nuevo.
6. Mapper: DTO nuevo → **las mismas entidades**.
7. Repositorio: nueva llamada y nuevos códigos de error.
8. Textos e íconos de la UI.
9. Tests con JSON reales.
10. Compilar, probar y revisar.